Hospital Patient Management System

Two-person implementation roadmap | React + Node/Express + MySQL

Handoff package: use this PDF together with *systemcontract.md*. The contract is the source of truth; this roadmap is the execution plan.

1. Project architecture

```
Browser
↓ HTTP / JSON
React + Vite frontend :5173
↓ REST API
Node.js + Express backend :5000
↓ mysql2/promise
MySQL 8 :3306
```

Frontend and backend are separate. The browser never talks directly to MySQL.

2. Final corrected data model

• Employee: Emp_ID, composite Name, composite Address, multivalued Mob_No, DOB, Salary. Age is derived and not stored.

• Employee ISA Doctor/Nurse: disjoint and partial. Doctor and Nurse inherit Emp_ID.

• Doctor: Department and Qualification. Department stays an attribute.

• Patient: P_ID, composite Name, Gender, DOB, multivalued Phone and Allergies. Age is not stored.

• Room: R_ID, Availability, Capacity, Type.

• Consults: Doctor M:N Patient with Date and Time.

• Assigned: Patient N:1 Room, using (P_ID, R_ID).

• Governs: Nurse M:N Room with Shift.

• Bills and TestReport remain weak entities with composite keys (P_ID, B_ID) and (P_ID, Test_ID).

3. Team split

Owner	Primary track	Deliverables
Person A	Track 1: Backend + SQL	MySQL schema/seed, Express API, controllers/services, validation, transactions, PDF service, backend tes
Person B	Track 2: Frontend	React pages, routing, components, forms/tables, API client, CRUD UX, dashboard
Both	Track 3: Integration	Contract review, end-to-end tests, bug fixes, README, GitHub and demo

4. Track 1 | Person A | Backend + SQL

4.1 Bootstrap

• Create backend/package.json. Install Express, mysql2, dotenv, cors, validation middleware and PDFKit/equivalent.

• Create src/app.js, src/server.js, src/config/db.js and GET /api/health.

4.2 MySQL

• Create backend/sql/schema.sql and seed.sql.

• Use MySQL 8.x and InnoDB. Define PKs/FKs explicitly and create indexes.

• Create tables in dependency order: employee, employee_phone, doctor, nurse, patient, patient_phone, patient_allergy, room, consults, assigned, governs, bills, test_report.

```
mysql -u root -p < sql/schema.sql
mysql -u root -p hospital_pms < sql/seed.sql
```

4.3 Connection

• Use mysql2/promise connection pool. Read DB_HOST, DB_PORT, DB_USER, DB_PASSWORD and DB_NAME from backend/.env.

```
const pool = mysql.createPool({
host: process.env.DB_HOST,
port: process.env.DB_PORT,
user: process.env.DB_USER,
password: process.env.DB_PASSWORD,
database: process.env.DB_NAME,
waitForConnections: true,
connectionLimit: 10
});
```

4.4 Backend files

```
backend/src/
■■■ config/db.js
■■■ controllers/
■ ■■■ employeeController.js
■ ■■■ doctorController.js
■ ■■■ nurseController.js
■ ■■■ patientController.js
■ ■■■ roomController.js
■ ■■■ consultationController.js
■ ■■■ assignmentController.js
■ ■■■ governsController.js
■ ■■■ billController.js
■ ■■■ testReportController.js
■■■ routes/
■■■ middleware/
■ ■■■ errorHandler.js
■ ■■■ validate.js
■■■ services/
■ ■■■ patientService.js
■ ■■■ pdfService.js
■■■ app.js
■■■ server.js
backend/sql/
■■■ schema.sql
■■■ seed.sql
```

4.5 CRUD build order

• First: Employee, Doctor, Nurse, Patient, Room.

• Second: patient phones/allergies, Consults, Assigned, Governs.

• Third: Bills and TestReport.

• Fourth: dashboard aggregate endpoints and patient PDF.

4.6 Database integrity

• Use parameterized SQL only. Validate input. Handle duplicate-key and foreign-key errors. Use transactions for multi-table operations.

• Patient creation should be one transaction: INSERT patient, INSERT phones, INSERT allergies, COMMIT; otherwise ROLLBACK.

4.7 Backend tests

• Test every GET/POST/PUT/DELETE endpoint. Test invalid IDs, duplicate IDs, missing parents, invalid dates, negative amounts and failed transactions.

5. Track 2 | Person B | Frontend

5.1 Bootstrap

```
cd frontend
npm create vite@latest . -- --template react
npm install
npm install react-router-dom axios
npm run dev
```

• Set VITE_API_BASE_URL=http://localhost:5000/api.

5.2 Frontend files

```
frontend/src/
■■■ api/
■ ■■■ client.js
■ ■■■ employees.js
■ ■■■ doctors.js
■ ■■■ nurses.js
■ ■■■ patients.js
■ ■■■ rooms.js
■ ■■■ consultations.js
■ ■■■ assignments.js
■ ■■■ governs.js
■ ■■■ bills.js
■ ■■■ testReports.js
■■■ components/
■ ■■■ Navbar.jsx
■ ■■■ Sidebar.jsx
■ ■■■ DataTable.jsx
■ ■■■ Modal.jsx
■ ■■■ ConfirmDialog.jsx
■ ■■■ FormField.jsx
■■■ pages/
```

```
■ ■■■ Dashboard.jsx
■ ■■■ Employees.jsx
■ ■■■ Doctors.jsx
■ ■■■ Nurses.jsx
■ ■■■ Patients.jsx
■ ■■■ PatientDetails.jsx
■ ■■■ Rooms.jsx
■ ■■■ Consultations.jsx
■ ■■■ Assignments.jsx
■ ■■■ Governs.jsx
■ ■■■ Bills.jsx
■ ■■■ TestReports.jsx
■■■ App.jsx
■■■ main.jsx
```

5.3 UI sequence

• Build reusable Navbar/Sidebar, DataTable, Modal, ConfirmDialog, FormField, loading states and error states.

• Build core pages: Employees, Doctors, Nurses, Patients, Rooms.

• Build relationship pages: Consultations, Assignments, Governs.

• Build Bills and Test Reports.

• Patient Details should aggregate phones, allergies, room, consultations, bills and test reports.

• Every major page must support Create, Read/List, Update and Delete.

5.4 API client

• Keep HTTP calls in src/api/*.js. Components should call functions such as getPatients(), createPatient(), updatePatient(), deletePatient().

5.5 CRUD UX

• Create: POST then refresh/update list. Edit: PUT then update row. Delete: confirmation then DELETE. Show backend validation errors.

6. Track 3 | Joint integration

6.1 Contract-first

• Both people read systemcontract.md. Person A freezes endpoint names and JSON shapes. Person B consumes exactly those shapes.

6.2 First end-to-end feature

```
React Patient Form
↓ JSON
POST /api/patients
↓
Express route
↓
controller
↓
patientService
↓
MySQL transaction
↓
JSON response
↓
React refreshes UI
```

• Make Patient CRUD the first complete vertical slice. Then copy the pattern to other modules.

6.3 Full-system test

• Create employee -> make doctor/nurse -> create patient with multiple phones/allergies -> create room -> assign patient -> create consultation -> create nurse-room shift -> create bill -> create test report -> edit -> delete where allowed.

• Verify that MySQL persists changes after browser refresh and that FK violations are handled safely.

7. REST modules

```
/api/employees
/api/doctors
/api/nurses
/api/patients
/api/patients/:pId/phones
/api/patients/:pId/allergies
/api/rooms
/api/consultations
/api/assignments
/api/governs
```

```
/api/bills
/api/test-reports
/api/health
```

Each major entity has GET list, GET one where appropriate, POST, PUT and DELETE. Composite-key resources receive all required key fields.

8. JSON contract

Patient create

```
{
"p_id": "P001",
"fname": "Rahul",
"mname": "K",
"lname": "Sharma",
"gender": "Male",
"dob": "2002-08-10",
"phones": ["9876543210"],
"allergies": ["Penicillin", "Dust"]
}
```

Bill

```
{
"p_id": "P001",
"b_id": "B001",
"amount": 18500
}
```

Test report

```
{
"p_id": "P001",
"test_id": "T001",
"test_type": "CBC",
"result": "Hemoglobin normal"
}
```

Response envelope

```
{
"success": true,
"data": {},
"message": "Operation successful"
}

{
"success": false,
"error": {
"code": "VALIDATION_ERROR",
"message": "Invalid request data",
"details": []
}
}
```

9. Milestone roadmap

Phase	Person A	Person B	Checkpoint
1 Foundation	MySQL + schema + seed + Express + health	Vite + routing + layout + API client	Both run locally
2 Core CRUD	Employee/Doctor/Nurse/Patient/Room APIs	Core entity pages	CRUD works
3 Relationships	Phones, allergies, consultations, assignments, governs	Patient details + relationship UI	FK behavior verified
4 Dependent entities	Bills + TestReport APIs	Bills + TestReport pages	Composite keys work
5 Advanced	Transactions + dashboard API + PDF service	Dashboard + filters + PDF button	Live DB report works
6 Hardening	Validation + errors + indexes + tests	UX errors + loading + responsive cleanup	Regression test passes
7 Delivery	Backend README/setup	Frontend README/setup	Clean clone runs

10. Local run

```
# Terminal 1
cd backend
npm install
npm run dev

# Terminal 2
cd frontend
npm install
npm run dev
```

```
Frontend: http://localhost:5173
Backend: http://localhost:5000
API: http://localhost:5000/api
```

11. GitHub structure

```
hospital-pms/
■■■ frontend/
■■■ backend/
■■■ docs/
■ ■■■ systemcontract.md
■ ■■■ project-roadmap.pdf
■■■ README.md
■■■ .gitignore
```

• Never commit .env, node_modules, passwords, secrets or real patient data. Commit .env.example, schema.sql, seed.sql, systemcontract.md and this PDF.

12. Definition of done

• MySQL schema builds and seed data loads.

• Backend connects through a MySQL pool.

• Frontend and backend run as separate processes.

• All major entities support CRUD.

• Phones and allergies work as multivalued data.

• Doctor/Nurse specialization works.

• Consultations, assignments and shifts work.

• Bills and Test Reports work with composite keys.

• Validation, error handling and FK integrity work.

• Dashboard uses live API data.

• Patient PDF report is generated from live database data, if included.

• A clean GitHub clone can be installed and run locally.

Antigravity handoff: Give it both files together. Tell it to implement the project against systemcontract.md and use the roadmap for sequencing and ownership. It should not redesign the schema unless explicitly instructed.